

House Price Prediction & Area Analysis Using Machine Learning

Muhammad Umer Afaq

Sp24-bcs-077

Marwa Sagheer

Sp24-bcs-058

—

Machine Learning

—

Adnan Karamat

House Price Prediction & Area Analysis Using Machine Learning

Project Type: Machine Learning Project

Language Used: Python

1. Introduction

Machine Learning is a branch of Artificial Intelligence that enables computers to learn patterns from data and make predictions or decisions without being explicitly programmed.

In the real estate industry, predicting house prices accurately is very important for:

- Buyers
- Sellers
- Property dealers
- Real estate companies

This project focuses on developing a Machine Learning system that can:

1. Predict house prices using Linear Regression.
2. Group houses into categories using K-Means Clustering.

The project is implemented using Python and beginner-friendly Machine Learning libraries.

2. Problem Statement

House prices vary depending on several factors such as:

- Area size
- Number of bedrooms
- Bathrooms
- Floors
- Location quality

People often face difficulty in estimating the correct price of a house.

Similarly, real estate companies also need a way to categorize houses into:

- Luxury houses
- Medium-budget houses
- Low-budget houses

Manual analysis is difficult and time-consuming.

Therefore, a Machine Learning-based solution is needed to:

- Predict house prices automatically.
- Analyze and categorize houses efficiently.

3. Proposed Solution

The proposed system uses two Machine Learning algorithms:

Algorithm	Purpose
Linear Regression	Predict house prices
K-Means Clustering	Group houses into categories

The system:

1. Loads the housing dataset.
2. Preprocesses the data.
3. Trains a Linear Regression model.
4. Predicts house prices.
5. Applies K-Means clustering.
6. Divides houses into groups.
7. Displays visualizations and results.

```

8. from google.colab import files
9.
10.     # Upload the file from your local system
11.     # This will open a file selection dialog
12.     uploaded = files.upload()
13.     # =====
14.     # STEP 1: Import Required Libraries
15.     # =====
16.
17.     import pandas as pd      # For loading and working with data
18.     import numpy as np      # For numerical calculations
19.     import matplotlib.pyplot as plt # For creating graphs and charts
20.
21.     # Sklearn - for Machine Learning algorithms
22.     from sklearn.linear_model import LinearRegression # Linear Regression model
23.     from sklearn.cluster import KMeans              # K-Means Clustering model
24.     from sklearn.model_selection import train_test_split # Split data into train/test
25.     from sklearn.metrics import r2_score            # To check model accuracy
26.
27.     print("All libraries imported successfully!")
28.     # =====
29.     # STEP 2: Load the Dataset
30.     # =====
31.
32.     # Load CSV file into a DataFrame (like a table)
33.     df = pd.read_csv('kc_house_data.csv')
34.
35.     # Show the first 5 rows of the dataset
36.     print("First 5 rows of the dataset:")
37.     print(df.head())
38.
39.     print("\n" + "="*50)
40.
41.     # Show the shape: (rows, columns)
42.     print(f"\nDataset Size: {df.shape[0]} rows and {df.shape[1]} columns")
43.
44.     print("\nColumn Names:")
45.     print(df.columns.tolist())
46.     # =====

```

```

47.     # STEP 3: Basic Dataset Information
48.     # =====
49.
50.     # Show data types and count of non-null values
51.     print("Dataset Information:")
52.     print(df.info())
53.
54.     print("\nBasic Statistics:")
55.     print(df[['price', 'sqft_living', 'bedrooms', 'bathrooms', 'floors']].describe())
56.     # =====
57.     # STEP 4: Data Preprocessing
58.     # =====
59.
60. # --- 4a. Select Only Useful Columns --- 61.     # Features we will use
to predict the price
62. feature_columns = ['sqft_living', 'bedrooms', 'bathrooms', 'floors']
63.
64.     # The value we want to predict
65.     target_column = 'price'
66.
67.     # Create a new small dataframe with only the columns we need
68.     selected_data = df[feature_columns + [target_column]]
69.
70. print("Selected Columns:") 71.
    print(selected_data.head())
72.
73.     print("\n" + "="*50)
74.
75. # --- 4b. Check for Missing Values --- 76. print("\nMissing Values in
Each Column:")
77.     print(selected_data.isnull().sum())
78.
79.     # Drop rows with missing values (if any)
80.     selected_data = selected_data.dropna()
81.     print(f"\nRows after removing missing values:
{len(selected_data)}")

```

```

82.
83.     print("\n Data preprocessing complete!")
84.     # =====
85.     # STEP 5: Separate Features (X) and Target (y)
86.     # =====
87.
88.     # X = Features (inputs) — what we give to the model
89.     X = selected_data[feature_columns]
90.
91.     # y = Target (output) — what we want to predict
92.     y = selected_data[target_column]
93.
94.     print("Features (X) - first 5 rows:")
95.     print(X.head())
96.
97.     print("\nTarget (y) - first 5 values:")
98.     print(y.head())
99.
100.    print(f"\nX shape: {X.shape}")
101.    print(f"\ny shape: {y.shape}")
102.    # =====
103.    # STEP 6: Split Data into Training and Testing Sets
104.    # =====
105. 106.    # Split: 80% for training, 20% for testing
107.    # random_state=42 ensures we get the same split every time we
run
108.    X_train, X_test, y_train, y_test = train_test_split(
109.        X, y,
110.        test_size=0.2,    # 20% for testing
111.        random_state=42    # Fixed seed for reproducibility
112.    )
113. 114.    print("Data Split Results:")

```

```

115.     print(f" Total samples   : {len(X)}")
116.     print(f" Training samples : {len(X_train)} (80%)")
117.     print(f" Testing samples  : {len(X_test)} (20%)")
118.     # =====
119.     # STEP 7: Train the Linear Regression Model
120.     # =====
121. 122.     # Create the Linear Regression model object
123.     lr_model = LinearRegression()
124. 125.     # Train (fit) the model using training data

126.     # The model learns the relationship between features and price
127.     lr_model.fit(X_train, y_train)
128.
129.         print(" Model trained successfully!")
130.
131.     # Show the learned coefficients (weights for each feature) 132.     print("\nModel
Coefficients (weights learned by model):")
133.         for feature, coef in zip(feature_columns, lr_model.coef_):
134.             print(f" {feature:15s}: {coef:,.2f}")
135.
136.     print(f"\nBias (Intercept): {lr_model.intercept_:.2f}")
137.     # =====
138.     # STEP 8: Predict House Prices
139.     # =====
140.
141.     # Use the trained model to predict prices on test data
142.     y_predicted = lr_model.predict(X_test)
143.
144.     # Compare actual vs predicted prices (first 10 results)
145.     print("Comparison: Actual Price vs Predicted Price")
146.     print("-" * 45)
147.     print(f"{'#':<5} {'Actual Price':>15} {'Predicted Price':>15}") 148.     print("-" * 45)
149.

```


PROJECT REPORT

```
150.         for i in range(10):
151.             actual = y_test.iloc[i]
152.             predicted = y_predicted[i]
153.             print(f'{i+1:<5} $ {actual:>14,.0f} $ {predicted:>14,.0f} ')
154.
155.     print("-" * 45)
156.     # =====
157.     # STEP 9: Check Model Accuracy using R2 Score
158.     # =====
159.
160.     # R2 (R-squared) Score:
161.     # - Value between 0 and 1
162.     # - Closer to 1 means better accuracy
163.     # - Example: 0.65 means model explains 65% of price variation
164.
165.     r2 = r2_score(y_test, y_predicted)
166.
167.     print("Model Accuracy Results:")
168.     print(f' R2 Score: {r2:.4f} ')
169.     print(f' Accuracy: {r2 * 100:.2f}%')
170.
171. # Interpretation
172.     if r2 >= 0.70:
173.         print(" Result : Good Model Accuracy!")
174.     elif r2 >= 0.50:
175.         print(" Result : Moderate Accuracy")
176.     else:
177.         print(" Result : Low Accuracy - needs improvement")
178.
179.     print("\nInterpretation:")
180.     print(f' The model can explain {r2*100:.1f}% of the variation in house prices.')
181.     # =====
182.     # STEP 10: Regression Visualization
183.     # =====
184.
185.     # Create a figure with 2 subplots side by side
186.     fig, axes = plt.subplots(1, 2, figsize=(14, 5))
187.     fig.suptitle('Linear Regression - House Price Prediction', fontsize=14, fontweight='bold')
188.
189.     # ----- Plot 1: Actual vs Predicted Prices -----
190.     axes[0].scatter(y_test, y_predicted, color='steelblue', alpha=0.4, s=15)
191.
```



```

192.     # Draw a perfect prediction line (diagonal)
193.     min_val = min(y_test.min(), y_predicted.min())
194.     max_val = max(y_test.max(), y_predicted.max())
195.     axes[0].plot([min_val, max_val], [min_val, max_val], color='red', linewidth=2, label='Perfect
    Prediction')

196.
197.     axes[0].set_xlabel('Actual Price ($)')
198.     axes[0].set_ylabel('Predicted Price ($)')
199.     axes[0].set_title('Actual Price vs Predicted Price')
200.     axes[0].legend()
201.     axes[0].grid(True, alpha=0.3)
202.
203.     # ----- Plot 2: sqft_living vs Price with Regression Line -----
204.     # Sample 500 points for cleaner plot
205.     sample = selected_data.sample(500, random_state=42)
206.
207.                                     axes[1].scatter(sample['sqft_living'], sample['price'],
208.                                     color='orange', alpha=0.5, s=20, label='House
    Data')

209.                                     # Draw a simple regression line for sqft_living vs price
210.                                     sqft_values = np.linspace(sample['sqft_living'].min(),
    sample['sqft_living'].max(), 100)

211.
212.     # Simple linear fit for visualization
213.     poly = np.polyfit(sample['sqft_living'], sample['price'], 1)
214.     price_line = np.polyval(poly, sqft_values)
215.
216.     axes[1].plot(sqft_values, price_line, color='red', linewidth=2, label='Regression Line')
217.     axes[1].set_xlabel('Living Area (sqft)')
218.     axes[1].set_ylabel('Price ($)')
219.     axes[1].set_title('Living Area vs Price')
220.     axes[1].legend()
221.     axes[1].grid(True, alpha=0.3)
222.
223.     plt.tight_layout()
224.     plt.show()
225.
226.     print(" Regression plots displayed!")
227.
228.     # =====

```

PROJECT REPORT

```
229.     # STEP 11: Prepare Data for K-Means Clustering
230.     # =====
231.
232.     # Select features for clustering
233.     cluster_features = ['price', 'sqft_living']
234.
235.     # Create a new dataframe with only clustering features
236.     cluster_data = df[cluster_features].dropna()
237.
238.     print("Clustering Data - first 5 rows:")
239.     print(cluster_data.head())
240.
241.     print(f"\nTotal rows for clustering: {len(cluster_data)}")
242.     print(f"Price range: ${cluster_data['price'].min():.0f} —
    ${cluster_data['price'].max():.0f}")
243.     print(f"Sqft range : {cluster_data['sqft_living'].min()} —
    {cluster_data['sqft_living'].max()} sqft")
244.     # =====
245.     # STEP 12: Apply K-Means Clustering
246.     # =====
247.
248.     # Create K-Means model with 3 clusters
249.     kmeans_model = KMeans(
250.         n_clusters=3,    # We want 3 groups
251.         random_state=42, # Fixed seed so results are same every time
252.         n_init=10        # Run 10 times, pick the best result
253.     )
254.
255.     # Fit the model — it finds the 3 groups automatically
256.     kmeans_model.fit(cluster_data)
257.
258.     # Get cluster label for each house (0, 1, or 2)
259.     cluster_labels = kmeans_model.labels_
260.
261.     # Add cluster labels back to the data
262.     cluster_data = cluster_data.copy()
263.     cluster_data['Cluster'] =
        cluster_labels
264.
265.     print(" K-Means Clustering complete!")
266.     print("\nCluster Labels (first 15 values):")
267.     print(cluster_labels[:15])
```

```

268.
269.     print("\nNumber of houses in each cluster:")
270.     print(cluster_data['Cluster'].value_counts().sort_index())
271.     # =====
272.     # STEP 13: Identify and Label Clusters (Budget / Medium / Luxury)
273.     # =====
274.
275.     # Get the centroid (center point) of each cluster
276.     centroids = kmeans_model.cluster_centers_
277.
278.     print("Cluster Centroids (Center Points):")
279.     print("-" * 45)
280.     print(f'{"Cluster":<10} {"Avg Price":>15} {"Avg Sqft":>12}')
281.     print("-" * 45)
282.     for i, centroid in enumerate(centroids):
283.         print(f'Cluster {i}   ${centroid[0]:>14,.0f}
           {centroid[1]:>11.0f}')
284.         print("-" * 45)
285.
286.     # -----
287.     # Automatically assign names based on average price
288.     # Cluster with highest avg price = Luxury
289.     # Cluster with lowest avg price = Budget
290.     # Middle one           = Medium
291.     # ----- 292.
293.     # Get average price for each cluster (index 0 = price column)
294.     avg_prices = {i: centroids[i][0] for i in range(3)}
295.
296.     # Sort clusters by average price
297.     sorted_clusters = sorted(avg_prices, key=avg_prices.get)
298.
299.     # Assign names
300.     cluster_names = {}
301.     cluster_names[sorted_clusters[0]] = 'Budget Houses'
302.     cluster_names[sorted_clusters[1]] = 'Medium Houses'
303.     cluster_names[sorted_clusters[2]] = 'Luxury Houses'
304.
305.     # Add house category name to data

```

PROJECT REPORT

```
306.     cluster_data['Category'] = cluster_data['Cluster'].map(cluster_names)
307.
308.     print("\nCluster Name Assignments:")
309.     for cluster_id, name in cluster_names.items():
310.         avg_p = avg_prices[cluster_id]
311.         print(f' Cluster {cluster_id} → {name:<20} (Avg Price: ${avg_p:,.0f})')
312.
313.     print("\nSample of clustered data:")
314.     print(cluster_data.head(10))
315.     # =====
316.     # STEP 14: Clustering Visualization
317.     # =====
318.
319.     # Define colors for each category
320.     color_map = {
321.         'Budget Houses' : 'green',
322.         'Medium Houses' : 'blue',
323.         'Luxury Houses' : 'red'
324.     } 325.
326.     # Create figure
327.     plt.figure(figsize=(10, 6))
328.
329.     # Plot each cluster with its own color
330.     for category, color in color_map.items():
331.         # Get rows belonging to this category
332.         group = cluster_data[cluster_data['Category'] == category]
333.         plt.scatter(
334.             group['sqft_living'], # X axis = Living Area
335.             group['price'],      # Y axis = Price
336.             c=color,
337.             label=category,
338.             alpha=0.4,
339.             s=10
340.         ) 341.
342.     # Plot the centroids as big star markers
343.     plt.scatter(
344.         centroids[:, 1], # sqft_living of centroids
345.         centroids[:, 0], # price of centroids
346.         c='black',
```

```

347.         marker='*',      # Star shape
348.         s=300,
349.         zorder=5,      # Draw on top
350.         label='Centroids'
351.     ) 352.
353.         # Add centroid labels
354.         for i, centroid in enumerate(centroids):
355.             name = cluster_names[i]
356.             plt.annotate(
357.                 name,
358.                 xy=(centroid[1], centroid[0]),
359.                 xytext=(centroid[1] + 100, centroid[0] + 50000),
360.                 fontsize=9,
361.                 color='black',
362.                 fontweight='bold'
363.             ) 364.
365.     # Labels and title
366.     plt.xlabel('Living Area (sqft)', fontsize=12)
367.     plt.ylabel('Price ($)', fontsize=12)
368.     plt.title('K-Means Clustering: House Categories\n(Budget / Medium / Luxury)', fontsize=13,
369.               fontweight='bold')
370.     plt.legend(fontsize=10)
371.     plt.grid(True, alpha=0.3)
372.     plt.tight_layout()
373.     plt.show() 373.
374.     print(" Clustering visualization displayed!")
375.     # =====
376.     # STEP 15: Cluster Summary (Bar Chart)
377.     # =====
378.
379.     # Count how many houses are in each category
380.     category_counts = cluster_data['Category'].value_counts()
381.
382.     # Define order for display
383.     order = ['Budget Houses', 'Medium Houses', 'Luxury Houses']
384.     counts = [category_counts.get(cat, 0) for cat in order]
385.     colors = ['green', 'blue', 'red']
386.

```

PROJECT REPORT

```
387.     # Create bar chart
388.     plt.figure(figsize=(7, 4))
389.     bars = plt.bar(order, counts, color=colors, width=0.5, alpha=0.8)
390.
391.     # Add count labels on top of each bar
392.     for bar,
count in zip(bars, counts):
393.         plt.text(
394.             bar.get_x() + bar.get_width() / 2,
395.             bar.get_height() + 100,
396.             str(count),
397.             ha='center', va='bottom', fontsize=11, fontweight='bold'
398.         )
399.         plt.xlabel('House Category', fontsize=11)
400.         plt.ylabel('Number of Houses', fontsize=11)
401.         plt.title('Number of Houses in Each Cluster', fontsize=12, fontweight='bold')
402.         plt.grid(axis='y', alpha=0.3)
403.         plt.tight_layout()
404.         plt.show()
405.
406.         print("House Count per Cluster:")
407.         for cat, count in zip(order, counts):
408.             print(f" {cat:<20}: {count} houses")
409.         # =====
410.         # STEP 16: Final Interpretation Summary
411.         # =====
412.
413.         print("=" * 60)
414.         print("    FINAL RESULTS SUMMARY")
415.         print("=" * 60)
416.
417.         print("\n LINEAR REGRESSION RESULTS")
418.         print("-" * 40)
419.         print(f" R2 Score : {r2:.4f} ({r2*100:.1f}% accuracy)")
420.         print(" Meaning : The model learned that bigger houses")
421.         print(" (more sqft) have higher prices.")
422.         print(" Top Feature: sqft_living has the most impact on price.")
423.
424.         print("\n K-MEANS CLUSTERING RESULTS")
425.         print("-" * 40)
426.         print(" 3 clusters were created automatically:")
```

```

427.
428.         for cat in ['Budget Houses', 'Medium Houses', 'Luxury Houses']:
429.             group = cluster_data[cluster_data['Category'] == cat]
430.             avg_p = group['price'].mean()
431.             avg_s = group['sqft_living'].mean()
432.             count = len(group)
433.             print(f"\n {cat}")
434.             print(f"   Houses : {count}")
435.             print(f"   Avg Price: ${avg_p:,.0f}") 436.         print(f"   Avg Size : {avg_s:.0f} sqft")
437.
438.     print("\n" + "=" * 60)
439.     print(" Both algorithms worked correctly on real house data!")
440.     print("=" * 60)
441.     # =====
442.     # BONUS: Predict Price for a Custom House
443.     # =====
444.
445. # Enter your own house details below and see the predicted price!
446.
447.     my_house_sqft    = 2000 # Living area in square feet
448.     my_house_bedrooms = 3    # Number of bedrooms
449.     my_house_bathrooms = 2   # Number of bathrooms
450.     my_house_floors   = 1    # Number of floors
451.
452.     # Put these values in the same format as training data
453.     my_house = pd.DataFrame({
454.         'sqft_living': [my_house_sqft],
455.         'bedrooms'   : [my_house_bedrooms],
456.         'bathrooms'  : [my_house_bathrooms],
457.         'floors'     : [my_house_floors]
458.     }) 459.
460.     # Predict price
461.     my_predicted_price = lr_model.predict(my_house)[0]
462.
463.     print(" My Custom House:")
464.     print(f"   Living Area : {my_house_sqft} sqft") 465.         print(f"   Bedrooms :
{my_house_bedrooms}")
466.     print(f"   Bathrooms : {my_house_bathrooms}")

```


PROJECT REPORT

```
467.     print(f" Floors    : {my_house_floors}")
468.     print(f"\n Predicted Price: ${my_predicted_price:,.0f}")
```

4. Objectives of the Project

The main objectives of this project are:

- To predict house prices accurately.
- To understand Machine Learning concepts.
- To implement Linear Regression practically.
- To implement K-Means Clustering practically.
- To analyze housing data visually.
- To build a beginner-friendly ML project.

5. Scope of the Project

This project can be used in:

- Real estate companies
 - Property websites
 - Housing analysis systems
 - Educational Machine Learning projects
- Future improvements can

include:

- Location maps
- Advanced prediction models
- Web applications
- User interfaces

6. Technologies Used

Technology	Purpose
Python	Programming Language
Pandas	Data handling
NumPy	Numerical operations
Scikit-learn	Machine Learning algorithms
Matplotlib	Data visualization
Jupyter Notebook	Project execution

7. Dataset Description

Dataset Used:

kc_house_data.csv

The dataset contains information about houses and their prices.

Important Features Used

Feature	Description
sqft_living	Living area size
bedrooms	Number of bedrooms
bathrooms	Number of bathrooms
floors	Number of floors
price	Actual house price

PROJECT REPORT

8. Machine Learning Algorithms Used

8.1 Linear Regression

Linear Regression is a supervised Machine Learning algorithm used for predicting numerical values.

In this project, it is used to predict:

House Prices

The algorithm learns the relationship between:

- Area size
- Bedrooms
- Bathrooms
- Floors and predicts the final price.

Formula

$$y=mx+b$$

Where:

- y = predicted value
- m = slope
- x = input feature
- b = intercept

8.2 K-Means Clustering

K-Means is an unsupervised Machine Learning algorithm.

It groups similar data points into clusters.

In this project, K-Means divides houses into:

- Luxury houses
- Medium-budget houses

- Budget houses

Working Steps

1. Select number of clusters ($K=3$)
2. Assign points to nearest cluster
3. Calculate centroids
4. Repeat until stable clusters are formed

9. Project Workflow

Load Dataset



Data Preprocessing



Feature Selection



Train/Test Split



Apply Linear Regression



Predict Prices



Apply K-Means Clustering



Generate Clusters



Visualization & Results

Load dataset:

PROJECT REPORT

```
from google.colab import files

# Upload the file from your local system
# This will open a file selection dialog
uploaded = files.upload()

... Choose Files kc_house_data.csv
kc_house_data.csv(text/csv) - 2515206 bytes, last modified: 5/19/2026 - 100% done
Saving kc_house_data.csv to kc_house_data.csv
```

Processing: (Load dataset)

```
# =====
# STEP 1: Import Required Libraries
# =====

import pandas as pd          # For loading and working with data
import numpy as np           # For numerical calculations
import matplotlib.pyplot as plt # For creating graphs and charts

# Sklearn - for Machine Learning algorithms
from sklearn.linear_model import LinearRegression # Linear Regression model
from sklearn.cluster import KMeans               # K-Means Clustering model
from sklearn.model_selection import train_test_split # Split data into train/test
from sklearn.metrics import r2_score              # To check model accuracy

print("All libraries imported successfully!")
```

... All libraries imported successfully!

```
# =====
# STEP 2: Load the Dataset
# =====

# Load CSV file into a DataFrame (like a table)
df = pd.read_csv('kc_house_data.csv')

# Show the first 5 rows of the dataset
print("First 5 rows of the dataset:")
print(df.head())

print("\n" + "="*50)

# Show the shape: (rows, columns)
print(f"\nDataset Size: {df.shape[0]} rows and {df.shape[1]} columns")

print("\nColumn Names:")
print(df.columns.tolist())
```

First 5 rows of the dataset:

	id	date	price	bedrooms	bathrooms	sqft_living	\
0	7129300520	20141013T000000	221900.0	3	1.00	1180	
1	6414100192	20141209T000000	538000.0	3	2.25	2570	
2	5631500400	20150225T000000	180000.0	2	1.00	770	
3	2487200875	20141209T000000	604000.0	4	3.00	1960	

```

First 5 rows of the dataset:
... 0 7129300520 20141013T000000 221900.0 3 1.00 1180
1 6414100192 20141209T000000 538000.0 3 2.25 2570
2 5631500400 20150225T000000 180000.0 2 1.00 770
3 2487200875 20141209T000000 604000.0 4 3.00 1960
4 1954400510 20150218T000000 510000.0 3 2.00 1680

sqft_lot floors waterfront view ... grade sqft_above sqft_basement \
0 5650 1.0 0 0 ... 7 1180 0
1 7242 2.0 0 0 ... 7 2170 400
2 10000 1.0 0 0 ... 6 770 0
3 5000 1.0 0 0 ... 7 1050 910
4 8080 1.0 0 0 ... 8 1680 0

yr_built yr_renovated zipcode lat long sqft_living15 \
0 1955 0 98178 47.5112 -122.257 1340
1 1951 1991 98125 47.7210 -122.319 1690
2 1933 0 98028 47.7379 -122.233 2720
3 1965 0 98136 47.5208 -122.393 1360
4 1987 0 98074 47.6168 -122.045 1800

sqft_lot15
0 5650
1 7639
2 8062
3 5000
4 8080

```

Feature selection:

```

# =====
# STEP 3: Basic Dataset Information
# =====

# Show data types and count of non-null values
print("Dataset Information:")
print(df.info())

print("\nBasic Statistics:")
print(df[['price', 'sqft_living', 'bedrooms', 'bathrooms', 'floors']].describe())

```

```

Dataset Information:
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 21613 entries, 0 to 21612
Data columns (total 21 columns):
#   Column          Non-Null Count  Dtype
---  -
0   id               21613 non-null  int64
1   date             21613 non-null  object
2   price            21613 non-null  float64
3   bedrooms         21613 non-null  int64
4   bathrooms        21613 non-null  float64
5   sqft_living      21613 non-null  int64
6   sqft_lot         21613 non-null  int64
7   floors           21613 non-null  float64
8   waterfront       21613 non-null  int64

```

PROJECT REPORT

```
... 8 waterfront      21613 non-null int64
     9 view          21613 non-null int64
    10 condition     21613 non-null int64
    11 grade         21613 non-null int64
    12 sqft_above    21613 non-null int64
    13 sqft_basement 21613 non-null int64
    14 yr_built      21613 non-null int64
    15 yr_renovated  21613 non-null int64
    16 zipcode       21613 non-null int64
    17 lat           21613 non-null float64
    18 long          21613 non-null float64
    19 sqft_living15 21613 non-null int64
    20 sqft_lot15    21613 non-null int64
dtypes: float64(5), int64(15), object(1)
memory usage: 3.5+ MB
None
```

Basic Statistics:

	price	sqft_living	bedrooms	bathrooms	floors
count	2.161300e+04	21613.000000	21613.000000	21613.000000	21613.000000
mean	5.400881e+05	2079.899736	3.370842	2.114757	1.494309
std	3.671272e+05	918.440897	0.930062	0.770163	0.539989
min	7.500000e+04	290.000000	0.000000	0.000000	1.000000
25%	3.219500e+05	1427.000000	3.000000	1.750000	1.000000
50%	4.500000e+05	1910.000000	3.000000	2.250000	1.500000
75%	6.450000e+05	2550.000000	4.000000	2.500000	2.000000
max	7.700000e+06	13540.000000	33.000000	8.000000	3.500000

```
# STEP 4: Data Preprocessing
# =====

# --- 4a. Select Only Useful Columns ---
# Features we will use to predict the price
feature_columns = ['sqft_living', 'bedrooms', 'bathrooms', 'floors']

# The value we want to predict
target_column = 'price'

# Create a new small dataframe with only the columns we need
selected_data = df[feature_columns + [target_column]]

print("Selected Columns:")
print(selected_data.head())

print("\n" + "="*50)

# --- 4b. Check for Missing Values ---
print("\nMissing Values in Each Column:")
print(selected_data.isnull().sum())

# Drop rows with missing values (if any)
selected_data = selected_data.dropna()
print(f"\nRows after removing missing values: {len(selected_data)}")
```

Document1 - Word

```

... Selected Columns:
   sqft_living  bedrooms  bathrooms  floors  price
0         1180         3         1.00    1.0  221900.0
1         2570         3         2.25    2.0  538000.0
2          770         2         1.00    1.0  180000.0
3         1960         4         3.00    1.0  604000.0
4         1680         3         2.00    1.0  510000.0

```

=====

Missing Values in Each Column:

```

sqft_living  0
bedrooms     0
bathrooms    0
floors       0
price        0
dtype: int64

```

Rows after removing missing values: 21613

Data preprocessing complete!

Separate feature X and Y:

```

... Features (X) - first 5 rows:
   sqft_living  bedrooms  bathrooms  floors
0         1180         3         1.00    1.0
1         2570         3         2.25    2.0
2          770         2         1.00    1.0
3         1960         4         3.00    1.0
4         1680         3         2.00    1.0

```

Target (y) - first 5 values:

```

0    221900.0
1    538000.0
2    180000.0
3    604000.0
4    510000.0

```

Name: price, dtype: float64

X shape: (21613, 4)

y shape: (21613,)

Split Data into Training and Testing Sets:

```

# =====
# STEP 6: Split Data into Training and Testing Sets
# =====

# Split: 80% for training, 20% for testing
# random_state=42 ensures we get the same split every time we run
X_train, X_test, y_train, y_test = train_test_split(
    X, y,
    test_size=0.2,      # 20% for testing
    random_state=42     # Fixed seed for reproducibility
)

print("Data Split Results:")
print(f"  Total samples   : {len(X)}")
print(f"  Training samples : {len(X_train)} (80%)")
print(f"  Testing samples  : {len(X_test)} (20%)")

```

```

... Data Split Results:
  Total samples   : 21613
  Training samples : 17290 (80%)
  Testing samples  : 4323 (20%)

```

Train the Linear Regression Model:

PROJECT REPORT

```
# Create the Linear Regression model object
lr_model = LinearRegression()

# Train (fit) the model using training data
# The model learns the relationship between features and price
lr_model.fit(X_train, y_train)

print(" Model trained successfully!")

# Show the learned coefficients (weights for each feature)
print("\nModel Coefficients (weights learned by model):")
for feature, coef in zip(feature_columns, lr_model.coef_):
    print(f" {feature:15s}: {coef:,.2f}")

print(f"\nBias (Intercept): {lr_model.intercept_:,.2f}")
```

Model trained successfully!

Model Coefficients (weights learned by model):

sqft_living	: 305.63
bedrooms	: -56,445.97
bathrooms	: 11,748.92
floors	: -1,103.49

Bias (Intercept): 70,857.03

Predict House Prices:

*** Comparison: Actual Price vs Predicted Price

#		Actual Price	Predicted Price
1	\$	365,000	\$ 501,959
2	\$	865,000	\$ 709,102
3	\$	1,038,000	\$ 1,024,470
4	\$	1,490,000	\$ 1,334,114
5	\$	711,000	\$ 708,045
6	\$	211,000	\$ 389,546
7	\$	790,000	\$ 694,388
8	\$	680,000	\$ 479,925
9	\$	384,500	\$ 417,695
10	\$	605,000	\$ 246,735

heck Model Accuracy using R^2 Score:

```

r2 = r2_score(y_test, y_predicted)

print("Model Accuracy Results:")
print(f" R² Score: {r2:.4f}")
print(f" Accuracy: {r2 * 100:.2f}%")

# Interpretation
if r2 >= 0.70:
    print(" Result : Good Model Accuracy!")
elif r2 >= 0.50:
    print(" Result : Moderate Accuracy")
else:
    print(" Result : Low Accuracy - needs improvement")

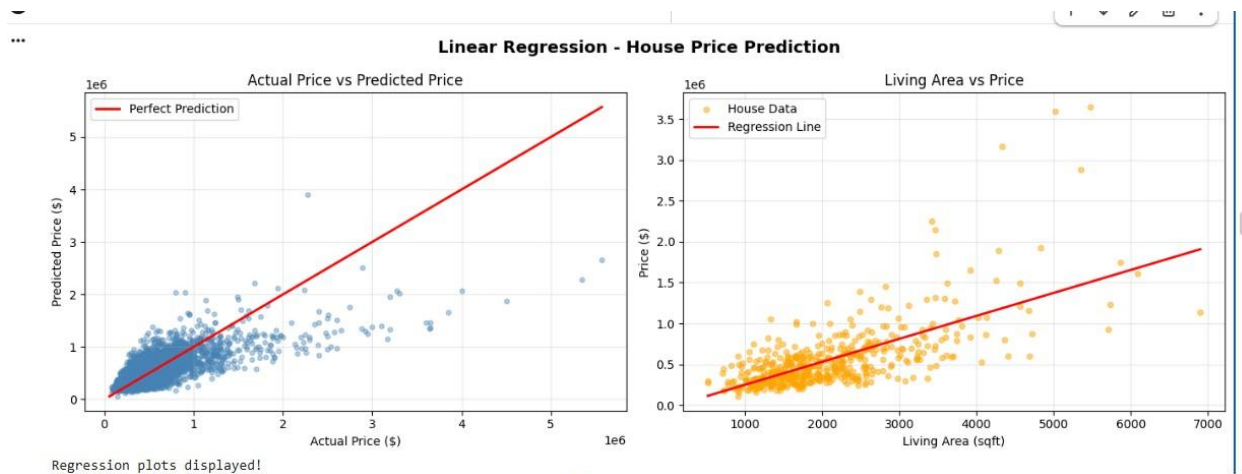
print("\nInterpretation:")
print(f" The model can explain {r2*100:.1f}% of the variation in house prices.")

```

Model Accuracy Results:
R² Score: 0.5089
Accuracy: 50.89%
Result : Moderate Accuracy

Interpretation:
The model can explain 50.9% of the variation in house prices.

Regression Visualization:



Prepare Data for K-Means Clustering:

PROJECT REPORT

```
# Select features for clustering
cluster_features = ['price', 'sqft_living']

# Create a new dataframe with only clustering features
cluster_data = df[cluster_features].dropna()

print("Clustering Data - first 5 rows:")
print(cluster_data.head())

print(f"\nTotal rows for clustering: {len(cluster_data)}")
print(f"Price range: ${cluster_data['price'].min():,.0f} - ${cluster_data['price'].max():,.0f}")
print(f"Sqft range : {cluster_data['sqft_living'].min()} - {cluster_data['sqft_living'].max()} sqft")
```

Clustering Data - first 5 rows:

	price	sqft_living
0	221900.0	1180
1	538000.0	2570
2	180000.0	770
3	604000.0	1960
4	510000.0	1680

Total rows for clustering: 21613
Price range: \$75,000 - \$7,700,000
Sqft range : 290 - 13540 sqft

Apply K-Means Clustering:

```
▶ # Get cluster label for each house (0, 1, or 2)
cluster_labels = kmeans_model.labels_

# Add cluster labels back to the data
cluster_data = cluster_data.copy()
cluster_data['Cluster'] = cluster_labels

print(" K-Means Clustering complete!")
print("\nCluster Labels (first 15 values):")
print(cluster_labels[:15])

print("\nNumber of houses in each cluster:")
print(cluster_data['Cluster'].value_counts().sort_index())
```

... K-Means Clustering complete!

Cluster Labels (first 15 values):
[1 1 1 0 1 0 1 1 1 1 0 1 1 1 1]

Number of houses in each cluster:
Cluster
0 5915
1 15094
2 604
Name: count, dtype: int64

Identify and Label Clusters (Budget / Medium / Luxury):

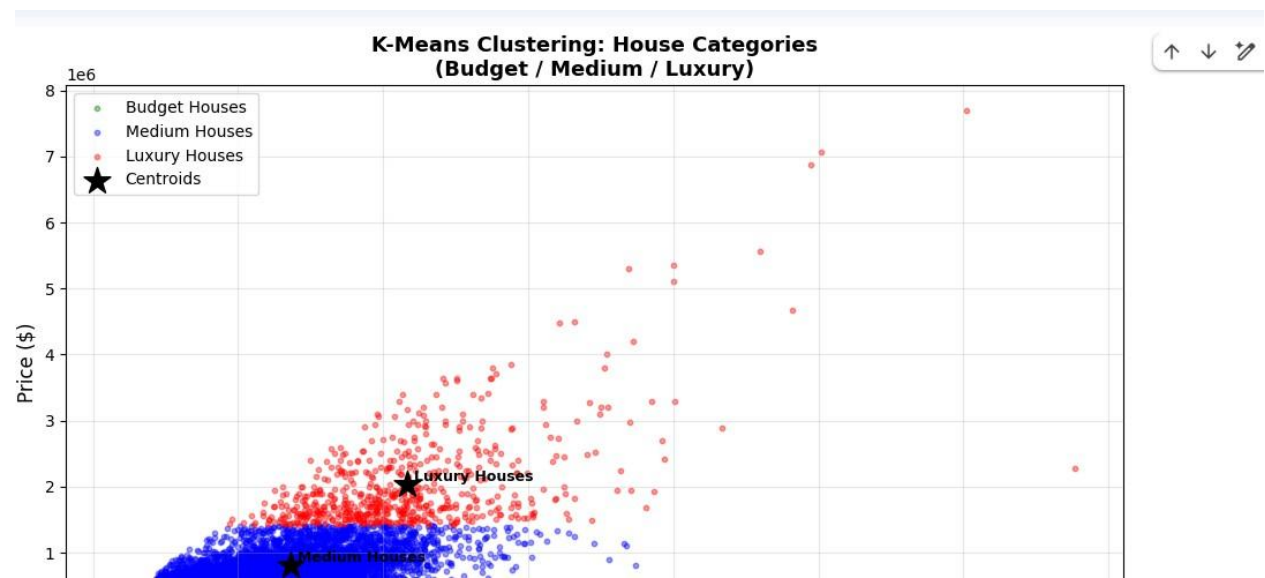
Cluster Centroids (Center Points):		

Cluster	Avg Price	Avg Sqft

Cluster 0	\$ 809,376	2723
Cluster 1	\$ 374,942	1738
Cluster 2	\$ 2,030,683	4333

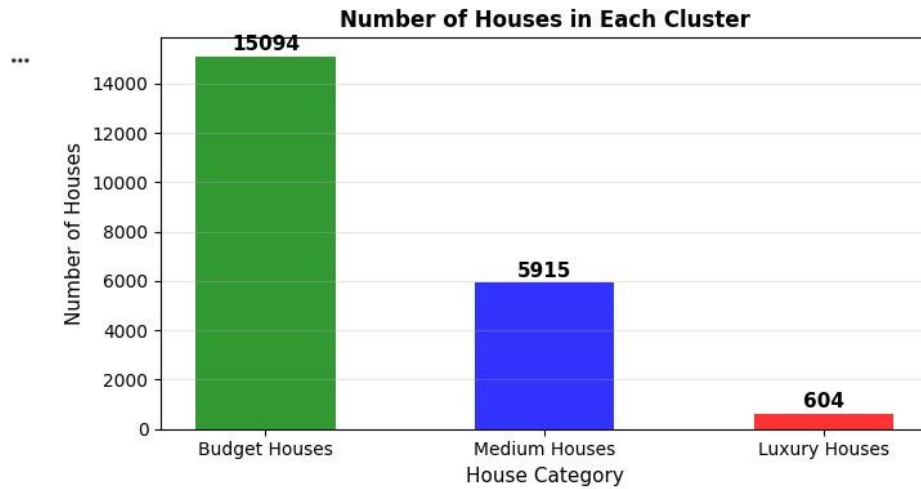
Cluster Name Assignments:		
Cluster 1	→ Budget Houses	(Avg Price: \$374,942)
Cluster 0	→ Medium Houses	(Avg Price: \$809,376)
Cluster 2	→ Luxury Houses	(Avg Price: \$2,030,683)
Sample of clustered data:		
	price	sqft_living
0	221900.0	1180
1	538000.0	2570
2	180000.0	770
3	604000.0	1960
4	510000.0	1680
5	1225000.0	5420
6	257500.0	1715
7	291850.0	1060
8	229500.0	1780

Clustering Visualization:



Cluster Summary (Bar Chart):

PROJECT REPORT



House Count per Cluster:
Budget Houses : 15094 houses
Medium Houses : 5915 houses
Luxury Houses : 604 houses

Final Interpretation Summary:

FINAL RESULTS SUMMARY
=====

...
LINEAR REGRESSION RESULTS

R^2 Score : 0.5089 (50.9% accuracy)
Meaning : The model learned that bigger houses
(more sqft) have higher prices.
Top Feature: sqft_living has the most impact on price.

K-MEANS CLUSTERING RESULTS

3 clusters were created automatically:

Budget Houses
Houses : 15094
Avg Price: \$374,927
Avg Size : 1738 sqft

Medium Houses
Houses : 5915
Avg Price: \$809,339
Avg Size : 2723 sqft

Luxury Houses
Houses : 604
Avg Price: \$2,030,683

Predict Price for a Custom House:

```

# Put these values in the same format as training data
my_house = pd.DataFrame({
    'sqft_living': [my_house_sqft],
    'bedrooms'    : [my_house_bedrooms],
    'bathrooms'   : [my_house_bathrooms],
    'floors'      : [my_house_floors]
})

# Predict price
my_predicted_price = lr_model.predict(my_house)[0]

print(" My Custom House:")
print(f"    Living Area : {my_house_sqft} sqft")
print(f"    Bedrooms    : {my_house_bedrooms}")
print(f"    Bathrooms    : {my_house_bathrooms}")
print(f"    Floors       : {my_house_floors}")
print(f"\n Predicted Price: ${my_predicted_price:,.0f}")

```

```

... My Custom House:
    Living Area : 2000 sqft
    Bedrooms    : 3
    Bathrooms    : 2
    Floors       : 1

Predicted Price: $535,177

Cluster Assignment: Cluster 1 → Budget Houses
This house falls in the 'Budget Houses' category

```

10. Data Preprocessing

Data preprocessing is an important step in Machine Learning.

The following preprocessing steps were performed:

- Loading dataset using Pandas
- Selecting useful columns
- Checking missing values
- Splitting features and target variables

Selected Features

$X = \text{data}[['\text{sqft_living}', '\text{bedrooms}', '\text{bathrooms}', '\text{floors}']]$

Target Variable $y = \text{data}['\text{price}']$

11. Train-Test Split

The dataset was divided into:

- Training Data = 80%

PROJECT REPORT

- Testing Data = 20% Purpose:
- Training data trains the model.
- Testing data evaluates performance. **Code**

```
from sklearn.model_selection import train_test_split
```

```
X_train, X_test, y_train, y_test = train_test_split(  
    X, y, test_size=0.2, random_state=42  
)
```

12. Linear Regression Implementation

Importing Model from sklearn.linear_model

import LinearRegression **Creating and Training**

Model model = LinearRegression()

model.fit(X_train, y_train) **Predictions** predictions
= model.predict(X_test)

Model Performance

R² score was used to evaluate model performance.

13. House Price Prediction Example Example Input

```
my_house = [[2000, 3, 2, 1]]
```

Predicted Output

Predicted House Price

This feature makes the project interactive and practical.

14. Regression Visualization

Scatter plots were used to visualize:

- Actual prices
- Predicted prices Purpose:
- Understand prediction trends
- Compare model performance visually

Visualization helps improve understanding of the dataset.

15. K-Means Clustering Implementation

Importing K-Means from sklearn.cluster import

KMeans Creating Model kmeans =

KMeans(n_clusters=3, random_state=42) **Training**

Model kmeans.fit(cluster_data) **Cluster Labels**

data['Cluster'] = kmeans.labels_

16. Cluster Categories

The houses were divided into:

- Cluster 0 → Budget Houses
- Cluster 1 → Medium Houses
- Cluster 2 → Luxury Houses

These categories help analyze housing patterns.

17. Clustering Visualization

Scatter plots were used to visualize:

- House groups
- Cluster boundaries

PROJECT REPORT

- Centroids

Centroids represent the center of each cluster.

Visualization improves project presentation and analysis.

18. Features of the System

Feature	Description
House Price Prediction	Predicts price using Linear Regression
House Categorization	Groups houses using K-Means
Data Visualization	Shows graphs and plots
Beginner-Friendly Code	Easy implementation

Feature	Description
Real-Life Dataset	Practical project
Prediction Example	User-defined prediction

19. Advantages of the Project

- Easy to understand
- Real-world application
- Beginner-friendly
- Good for Machine Learning practice
- Attractive visualizations
- Practical implementation of ML concepts

20. Limitations of the Project

- Limited features used
- Location information simplified
- Prediction accuracy depends on dataset quality
- Does not include deep learning techniques

21. Future Enhancements

Future improvements may include:

- Adding GUI interface
- Using more advanced algorithms
- Adding map-based location analysis
- Creating a web application
- Improving prediction accuracy

22. Conclusion

This project successfully implemented:

- Linear Regression for house price prediction
- K-Means Clustering for area analysis The system was able to:
- Predict house prices
- Group houses into meaningful categories
- Visualize housing data effectively

The project demonstrates how Machine Learning can solve real-world problems in the real estate industry.

PROJECT REPORT

It is beginner-friendly, practical, and useful for understanding basic Machine Learning concepts.

23. Required Libraries

```
pip install pandas  
pip install numpy  
pip install matplotlib  
pip install scikit-learn
```

24. Expected Outputs

The system produces:

- Predicted house prices
- Cluster labels
- Scatter plots
- Regression analysis
- House categories

25. Final Result

The project successfully:

- Predicted house prices using Linear Regression.
- Grouped houses using K-Means Clustering.
- Demonstrated practical Machine Learning implementation using Python.